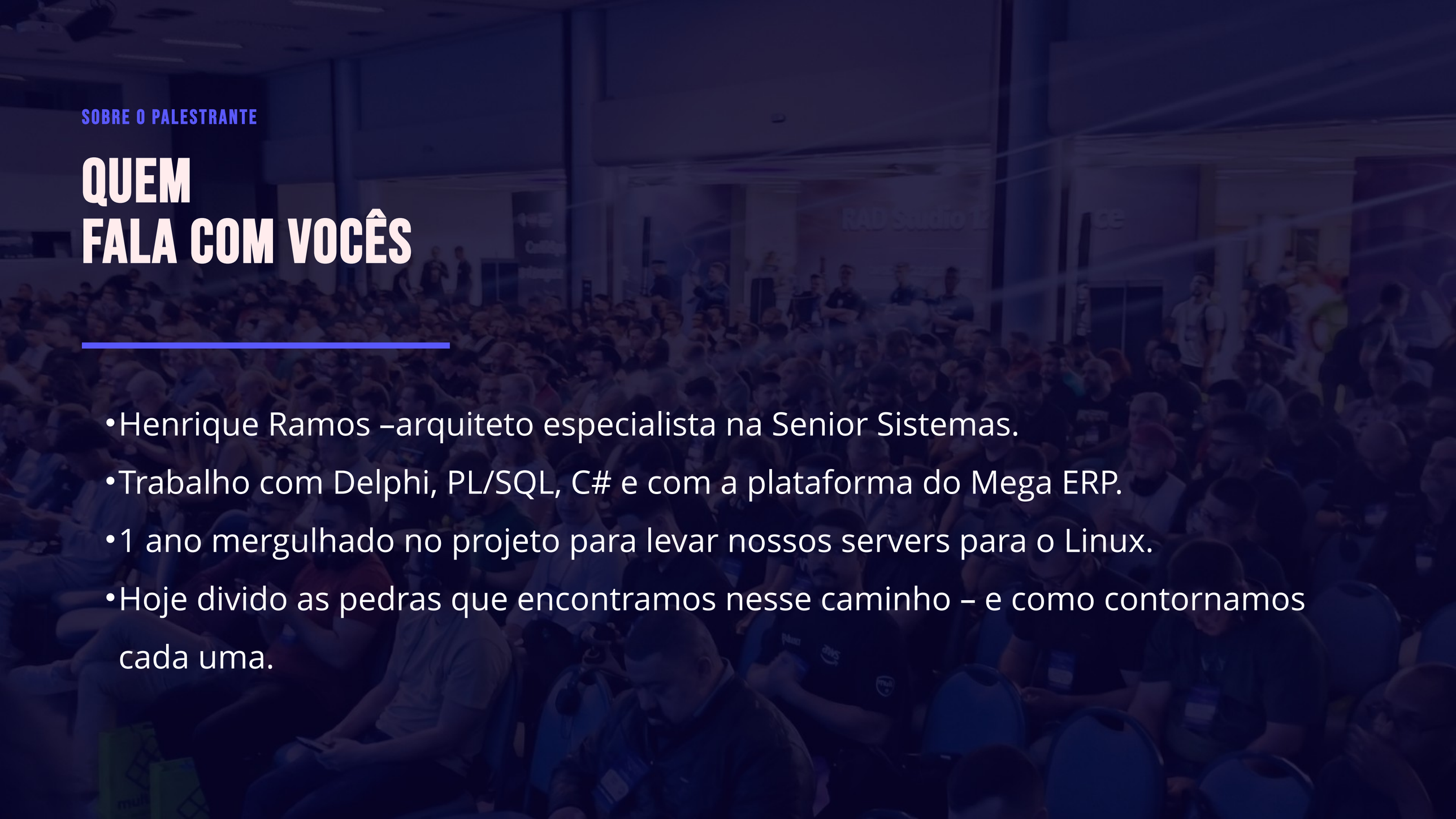

EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

OS DESAFIOS NO USO DO LINUX

Como levamos os servers de um ERP com mais de 20 anos e que só rodavam em Windows para também executar em Linux, e as pedras que encontramos no caminho.



SOBRE O PALESTRANTE

QUEM FALA COM VOCÊS

- Henrique Ramos –arquiteto especialista na Senior Sistemas.
- Trabalho com Delphi, PL/SQL, C# e com a plataforma do Mega ERP.
- 1 ano mergulhado no projeto para levar nossos servers para o Linux.
- Hoje divido as pedras que encontramos nesse caminho – e como contornamos cada uma.

O QUE VEREMOS

AGENDA

- De onde partimos: os servers do Mega e a arquitetura Client/Server.
- Por que Linux? A motivação por trás do projeto.
- Os desafios técnicos, um a um.
- Como preparamos o ambiente de debug no Linux.
- Dicas da comunidade, resultados e lições aprendidas.

CONTEXTO

O PONTO DE PARTIDA

Os servers do Mega ERP são aplicações headless em Delphi, no modelo Client/Server com RemObjects.

- Cada conexão instancia um Remote DataModule próprio.
- Acesso a dados Oracle via ODAC, com pooling de sessões escrito por nós.
- Regras de negócio distribuídas entre Delphi e PL/SQL.
- Um detalhe decisivo: eles já eram console ({$APPTYPE CONSOLE}) – mas só rodavam em Windows.

MOTIVAÇÃO

POR QUE LINUX?

- Redução de custo de licenciamento de sistema operacional.
- Portas abertas para containers e orquestração (Docker, Kubernetes).
- Aderência a ambientes de nuvem e a exigências de clientes.
- Servers headless são candidatos naturais: não dependem de interface gráfica.
- Maior resiliência.

O OBJETIVO NUNCA FOI TROCAR DE PLATAFORMA – FOI PASSAR A SUPORTAR AS DUAS.

VISÃO GERAL

FRENTES DO PROJETO

- Compilação cross-platform e a diretiva de plataforma.
- Sistema de arquivos: paths, barras e case-sensitivity.
- APIs exclusivas de Windows.
- Acesso a dados no Linux.
- Serviço do Windows virando daemon.
- Encoding, fim de linha e fontes.

SEM INTERFACE VISUAL

DESAFIO 1
SÓ CONSOLE

Nossos servers são aplicações headless (console)

- Mais complicado para realizar debugs
- Memory leaks causam grandes problemas em virtude da quantidade de processamento
- Foi preciso caçar usos indiretos de units visuais (TForm, TApplication gráfico).

REGRA DE OURO: NADA DE CLASSES VISUAIS EM CÓDIGO DE SERVER.

SISTEMA DE ARQUIVOS

DESAFIO 2
ARQUIVOS E PATHS

- Windows usa barra invertida; Linux usa barra normal. Concatenar caminho na mão quebra.

Solução: TPath, PathDelim e funções de path da plataforma.

- Linux é case-sensitive: Config e config são pastas diferentes.
- Nada de caminhos absolutos estilo c:\Versoes no código.

```
TPath.Combine(Base, 'Config')
```

CHAMADAS DE PLATAFORMA

DESAFIO 3
APIS DO WINDOWS

- Registro do Windows, unit Windows, mensagens e handles não existem no Linux.
- ShellExecute, WinAPI de serviço e afins precisaram de alternativas.

Estratégia: isolar o específico de plataforma com IFDEF MSWINDOWS / IFDEF LINUX.

- Onde possível, trocar por units multiplataforma (System.*, Posix.*).

BANCO NO LINUX

DESAFIO 4
ACESSO A DADOS

- As bibliotecas cliente do banco precisam existir no Linux (.so).
- O driver e o vendor lib devem ser resolvidos em runtime.

Sessões Oracle, cache de statements e thread-safety tiveram que ser revalidados fora do Windows.

CONECTAR DO WINDOWS E DO LINUX NÃO A MESMA CONFIGURAÇÃO.

EXECUÇÃO EM PRODUÇÃO

DESAFIO 5
SERVIÇO VIRA DAEMON

No Windows, o server roda como Serviço. No Linux, o equivalente é um daemon gerenciado por systemd.

- systemd cuida de start no boot, restart em falha e logs via journalctl.
- O loop principal (ProcessMessages/Terminated) precisou conviver com o modelo Linux.
- Timers baseados em componente Windows deram lugar a threads / timer multiplataforma.

ENCODING, EOL E FONTES

DESAFIO 6
DETALHES FINOS

- Fim de linha: CRLF (Windows) x LF (Linux) em arquivos de configuração e scripts.
- Encoding: garantir UTF-8 na leitura e escrita.
- Permissões de arquivo (chmod +x) para executáveis e scripts.
- Fontes e recursos que só existiam no Windows.

DEBUG NO LINUX

PREPARANDO O DEBUG

Debugar um server rodando no Linux, a partir do Delphi no Windows, exige o PAServer.

- PAServer: o agente que roda no Linux e conversa com a IDE.
- Uma VM (ou WSL2) com o ambiente já preparado.
- O projeto configurado para o profile de conexão Linux.

A SEGUIR, AS LIMITAÇÕES PARA O DEBUG

DEBUG NO LINUX

LIMITAÇÕES

- Precisa se acostumar com a pasta Scratch-dir.
- Não é possível visualizar objetos complexos.
- Access Violation faz com que a aplicação seja finalizada.
- EurekaLog não funciona no Linux

DEBUG NO LINUX

PASSO 2
CONFIGURAR O DELPHI

- Adicione o pacote FMXLinux pelo GetIt Package Manager.
- Edite a conexão do projeto pelo Project Manager.
- No Connection Profile Manager, informe o IP do Linux e teste a conexão.
- No SDK Manager, confirme os dados do Linux.
- Compile a aplicação.

DEBUG NO LINUX

DOCUMENTAÇÃO DA EMBARCADERO

Desenvolvimento para Linux

https://docwiki.embarcadero.com/RADStudio/Athens/en/Linux_Application_Development

Configurar ambiente para debug

https://blogs.embarcadero.com/deploying-firemonkey-to-html5-with-fmxlinux-on-wsl2/

O QUE APRENDEMOS COM OS OUTROS

DICAS DA COMUNIDADE

- Use WSL2, nunca WSL1: o debug só funciona com o kernel completo do WSL2.
- Em WSL, o IP do PAServer é localhost/127.0.0.1 – host e WSL compartilham portas.
- PAServer roda restrito: por padrão só copia para o scratch-dir.
- I/O Error 11 ao subir o PAServer aparece em algumas distros – vale checar a distro.

O QUE FICOU

RESULTADOS E LIÇÕES

- Os servers passaram a compilar e rodar também em Linux, sem perder o Windows.
- Código mais limpo: o específico de plataforma ficou isolado.
- Ambiente de debug documentado e replicável pelo time.
- Para nós é melhor usar VM do que WSL.

O MAIOR DESAFIO NÃO FOI O DELPHI – FOI A MUDANÇA DE MENTALIDADE SOBRE A PLATAFORMA.

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero® An Idera, Inc. Company



QUER ME VER NA
#ECON27?
ACESSE O QRCODE E
AVALIE MINHA PALESTRA!

HENRIQUE RAMOS

@henriqueframos

henriqueframos

henrique.ramos@gmail.com

(15) 9 9125-9469